

FRAMEWORKS DE IA — UNIDAD 3: MLOPS

ONNX: Open Neural Network Exchange

El formato universal para desplegar modelos de IA en producción

Estandarización de modelos de Machine Learning e Interoperabilidad entre PyTorch, TensorFlow, ONNX Runtime y Aceleradores Hardware.

 [Introducción](#)

 [PyTorch → ONNX](#)

 [TensorFlow → ONNX](#)

 [ONNX Runtime](#)

 [Recursos](#)

Open Neural Network Exchange es un formato de archivo abierto que permite representar modelos de ML independientemente del framework con que fueron creados. Funciona como un lenguaje común entre frameworks y runtimes.

Interoperabilidad

Un modelo entrenado en PyTorch corre en TensorFlow Serving, CoreML (iOS) o TensorRT (NVIDIA GPU) sin reescribir código.

Portabilidad

El archivo .onnx contiene toda la arquitectura y los pesos. Se mueve entre plataformas como cualquier archivo binario.

Optimización

ONNX Runtime aplica optimizaciones de grafo automáticas: fusión de operadores, cuantización y compilación JIT.

Línea de Tiempo y Adopción: **2017** Creado por Microsoft y Facebook (Meta) | **2019** Donado a la Linux Foundation | **Hoy** Soportado por más de 40 organizaciones líderes de la industria.

✗ SIN ONNX (Ecosistema Fragmentado)

PyTorch → Requiere Runtime de PyTorch

TensorFlow → Requiere Runtime de TF

MXNet / JAX → Requiere Runtimes específicos

Consecuencia: Cada framework necesita su propio entorno de inferencia, generando bloqueo de proveedor y redundancia.

✓ CON ONNX (Formato Único Universal)

Cualquier Framework exporta a ONNX y se despliega directamente en:

✓ **ONNX Runtime** (Servidores / Cloud)

✓ **CoreML** (iOS / macOS)

✓ **TensorRT** (NVIDIA GPU)

✓ **Web Assembly / ONNX.js** (Navegador web)

Ventaja: Un único formato de intercambio para cualquier runtime de producción.

Un modelo ONNX es un grafo computacional serializado en **Protocol Buffers** (.onnx = archivo binario eficiente).

1. Graph (Grafo)

Contiene la estructura completa:

- **Nodos:** Operaciones (Conv, ReLU...)
- **Edges:** Tensores que fluyen
- **Inputs / Outputs:** Definición global

2. Nodes (Operaciones)

Cada nodo define un `op_type`:

- Atributos: `kernel_size`, `stride`
- Especifican el cómputo abstracto sin datos concretos

3. Initializers (Pesos)

Parámetros entrenados del modelo:

- Pesos y biases numéricos fijos
- Se almacenan dentro del .onnx o externamente

 **Opset Versioning:** Cada versión de ONNX define un conjunto de operadores (*opset*). Al exportar se selecciona `opset_version` (por defecto versión 17). Versiones más altas admiten mayor variedad de operadores.

Exportar Modelo PyTorch

```
import torch import torch.nn as nn # 1. Definir y evaluar modelo modelo = nn.Sequential( nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2) ).eval() # 2.
```


Exportación e Inferencia en CNN

```
import torch, onnxruntime as ort from PIL import Image import torchvision.transforms as T # Input de imagen 224x224 RGB dummy = torch.randn(1, 3, 224
```



Comparativa de Rendimiento CPU:

- PyTorch Nativo: ~12 ms / imagen
- ONNX Runtime: ~6 ms / imagen (~2× más rápido)

Conversión desde Keras en Memoria

```
import tensorflow as tf import tf2onnx # 1. Definir modelo Keras model = tf.keras.Sequential([ tf.keras.layers.Input(shape=(10,)), tf.keras.layers.De
```



```
import tensorflow as tf import tf2onnx, onnxruntime as ort, numpy as np # 1. Cargar MobileNetV3 Preentrenado base = tf.keras.applications.MobileNe
```

Contenido del archivo .onnx exportado:

El archivo independiente incluye la **arquitectura completa**, los **pesos entrenados en ImageNet** y la especificación de operadores para su despliegue en cualquier servidor de producción.

ORT aplica optimizaciones automáticas al grafo computacional para maximizar la velocidad en el hardware objetivo mediante **Execution Providers**.

CPU & GPU NVIDIA

CPUExecutionProvider:

CPU por defecto
(AVX2/AVX512).

CUDAExecutionProvider:

GPU NVIDIA general.

TensorrtExecutionProvider:

Máximo rendimiento con
NVIDIA TensorRT.

Apple & Intel

CoreMLExecutionProvider:

Apple Silicon (Neural Engine
M1/M2/M3).

OpenVINOExecutionProvider:

Optimizado para
CPU/GPU/VPU Intel.

Selección Dinámica

```
# Selección de Providers sess = ort.InferenceSession( 'modelo.onnx', providers=[ 'Tensorrt'
```

Optimización en Cascada: ONNX Runtime evalúa los providers en orden y utiliza la opción con mayor aceleración disponible en la máquina.

Criterio	PyTorch → ONNX	TensorFlow → ONNX
Herramienta	<code>torch.onnx.export()</code> (Nativo)	<code>tf2onnx</code> (Librería separada)
Mecanismo	Tracing (Ejecuta forward pass)	Grafo estático (SavedModel / Keras)
Input de Ejemplo	Obligatorio (<code>dummy_input</code>)	No requerido (Solo firma TensorSpec)
Control de Flujo	Requiere <code>torch.jit.script()</code>	Soportado automáticamente
Modelos Complejos	Muy directo en redes de visión/NLP	A veces requiere simplificar nodos Keras
Resultado / Inferencia	Archivo <code>.onnx</code> ejecutable en ORT	Archivo <code>.onnx</code> ejecutable en ORT

Documentación Oficial

Sitio web oficial con especificación de operadores y tutoriales:

<https://onnx.ai> | <https://onnxruntime.ai>

Netron Graph Visualizer

Herramienta visual imprescindible para inspeccionar nodos, inputs y pesos de archivos .onnx:

<https://netron.app>

ONNX Model Zoo

Repositorio oficial con modelos preentrenados y optimizados en formato ONNX (ResNet, BERT, YOLO...):

github.com/onnx/models

Despliegue en Producción

Integración en servidores de inferencia escalables:

- Triton Inference Server (NVIDIA)
- Azure ML Endpoints & AWS SageMaker

☰ Pasos Clave antes del Despliegue:

- ✓ **Modo Evaluación:** Asegurar `model.eval()` antes de exportar PyTorch.
- ✓ **Elegir Opset Adecuado:** Usar `opset_version >= 17` para soporte moderno de operadores.
- ✓ **Dimensiones Dinámicas:** Configurar `dynamic_axes` para permitir lotes (*batch_size*) variables.
- ✓ **Validar Grafo:** Ejecutar `onnx.checker.check_model()` tras la exportación.

¿Preguntas o Dudas?

Ingeniería Civil Informática — Electivo 4° año — Unidad 3: MLOps

Image Sources



https://static.vecteezy.com/system/resources/previews/013/446/233/non_2x/ai-artificial-intelligence-concept-digital-technology-banner-pink-blue-background-deep-learning-machine-learning-abstract-tech-data-analysis-neural-network-world-future-illustration-vector.jpg

Source: www.vecteezy.com